

Master Roadmap for Designing an Autonomous ROS 2 Robot

Mobile base + robot arms + camera + LiDAR + embedded control + simulation + OTA development

Prepared for a robot whose motors, compute, microcontrollers, battery, arm, sensors, and body design are not yet fixed.
Generated June 25, 2026. Source register included at the end.

Core recommendation	Design software interfaces before committing to hardware. Make ROS 2 the high-level integration layer, keep hard real-time and safety loops on microcontrollers, and build a digital twin early.
Default software path	Use ROS 2 Lyrical/Ubuntu 26.04 if your required drivers support it. Use ROS 2 Jazzy/Ubuntu 24.04 if you want a mature LTS baseline with wider vendor driver availability today.
Default hardware path	Start with a differential-drive base, wheel encoders, IMU, 2D LiDAR, front RGB-D/stereo camera, one Linux compute unit, one motor-control MCU, and one supported off-the-shelf arm if manipulation matters.
Development path	Develop from Mac or Windows through VS Code Remote-SSH, Docker/dev containers, WSL2 on Windows when useful, and Foxglove/browser tools. The robot itself should run Linux.

This document is intentionally practical: it gives you architecture, software packages, hardware decision matrices, formulas, launch patterns, simulation workflow, firmware strategy, and phased build milestones. It is not a certification document; safety-critical work needs professional review.

Table of Contents

1. How to use this roadmap	7
1.1 Assumptions behind this guide	7
1.2 The artifact stack you should create before buying hardware	7
1.3 Immediate working rule	7
2. Define the robot before choosing parts	7
2.1 Mission profile questions	8
2.2 A starter requirements matrix	8
2.3 Traceability: combine academia's V-model discipline with ROS artifacts	8
3. System architecture: the master separation of responsibility	9
3.1 What belongs on microcontrollers	9
3.2 What belongs in ROS 2 on Linux	9
3.3 Recommended data/control split	10
4. ROS 2 distribution, operating system, and compatibility	10
4.1 Distro decision table	10
4.2 My recommended starting baseline	10
4.3 Mac and Windows development compatibility	11
4.4 Recommended developer environment	11
5. ROS 2 concepts you should design around now	11
5.1 Interface selection	12
5.2 QoS policy defaults for your robot	12
5.3 tf2 frame tree: define this before CAD is done	12
5.4 ros2_control and micro-ROS in the architecture	13
6. Mechanical platform and drive-base selection	13
6.1 Drive-base decision matrix	13
6.2 Default mechanical recommendation	13
6.3 Torque and speed sizing formulas	13

6.4 Chassis and mounting plan	14
7. Motors, motor drivers, encoders, and low-level motion	14
7.1 Motor type comparison	14
7.2 Drive controller architecture	15
7.3 Encoder recommendations	15
7.4 Motor driver selection criteria	15
8. Power system and battery design	15
8.1 Power tree template	15
8.2 Battery sizing formula	16
8.3 Battery chemistry choice	16
8.4 Power design checklist	16
9. Low-level hardware model and microcontroller architecture	16
9.1 Minimum MCU responsibilities	16
9.2 MCU and bus selection	17
9.3 micro-ROS versus custom protocol	17
9.4 Firmware state machine	17
9.5 Firmware update and OTA strategy	18
10. Main compute architecture	18
10.1 Compute options	18
10.2 Default compute recommendation	18
10.3 Onboard compute checklist	18
10.4 GPU and AI acceleration	19
11. Sensors and mounting strategy	19
11.1 LiDAR choices	19
11.2 Camera choices	19
11.3 IMU, wheel odometry, and sensor fusion	19
11.4 Mounting and calibration rules	20
12. Robot arm strategy	20

12.1 Arm category decision table	20
12.2 Arm mounting on a mobile base	20
12.3 MoveIt 2 architecture	20
12.4 Mobile manipulation rule	21
13. ROS 2 workspace, package layout, and naming conventions	21
13.1 Recommended repository layout	21
13.2 Naming conventions	21
13.3 Interface contract template	22
14. ROS 2 graph for the first complete robot	22
14.1 Core node list	22
14.2 Startup sequence	23
14.3 Lifecycle and fault state	23
15. Modeling the robot: URDF, Xacro, SDF, and ros2_control	23
15.1 URDF/Xacro modeling workflow	23
15.2 CAD-to-URDF rule	23
15.3 Minimal URDF/Xacro fragment	24
15.4 ros2_control simulation and hardware swap	24
16. Simulation and digital twin workflow	24
16.1 Simulator choice	24
16.2 Simulation levels	25
16.3 Digital twin development sequence	25
16.4 What simulation cannot guarantee	25
17. Navigation stack: odometry, SLAM, localization, and Nav2	25
17.1 Navigation architecture	26
17.2 Map workflow	26
17.3 Tuning priorities	26
17.4 Behavior trees	26

18. Manipulation stack: MoveIt 2, grippers, and perception	26
18.1 MoveIt integration checklist	27
18.2 Gripper design	27
18.3 Mobile manipulator coordination	27
18.4 Avoid early complexity	27
19. Perception and AI pipeline	27
19.1 Perception layers	28
19.2 Bandwidth controls	28
19.3 AI model deployment	28
20. Logging, observability, and debugging	28
20.1 Bag recording profiles	29
20.2 Operator dashboards	29
20.3 Diagnostics contract	29
21. Development operations, deployment, and OTA	29
21.1 Release model	29
21.2 Systemd bringup pattern	30
21.3 Docker and dev containers	30
21.4 OTA options	30
21.5 Remote access	30
22. Safety, reliability, and failure-mode design	30
22.1 Minimum safety architecture	31
22.2 Failure modes and controls	31
22.3 Commissioning rules	31
23. Build roadmap: from blank slate to autonomous mobile manipulator	31
23.1 Phase plan	32
23.2 First 30 days	32
24. A concrete starter configuration	32
24.1 Why not start with everything at once?	33

A. Appendix: hardware scoring matrix	33
B. Appendix: software templates	34
B.1 Top-level launch shape	34
B.2 Udev naming rule example	34
B.3 Bag recording command examples	34
C. Appendix: calculations worksheet	34
D. Appendix: commissioning checklists	35
D.1 Before applying battery power	35
D.2 Before first autonomous drive	35
D.3 Before arm motion on base	35
E. Appendix: research trail and source register	35
E.1 Search terms to continue research	36

1. How to use this roadmap

You are at the best possible point to think about ROS 2: before buying hardware. The decisions that most often cause expensive rework are not the brand of motor or the exact camera model; they are unclear interfaces, missing coordinate frames, weak power design, late simulation, and firmware that cannot be tested outside the robot.

The key idea is to make a software and systems model that is detailed enough to compare hardware options before buying them. Your first concrete design output should not be a shopping cart. It should be a folder containing requirements, a frame tree, a robot description model, interface definitions, a power budget, and a simulation launch file.

Think of the robot as a distributed computer attached to a machine. The machine is mechanical and electrical. The computer is split across Linux, microcontrollers, motor drives, sensors, and possibly a GPU. ROS 2 is the integration layer that keeps these pieces observable, replaceable, and testable.

1.1 Assumptions behind this guide

- The robot is a ground robot, not an aircraft. It will drive itself using a mobile base.
- It may carry one or more arms and at least one camera and LiDAR.
- You want meaningful simulation before all hardware is purchased.
- You may develop on Mac or Windows, but the onboard robot computer should run Linux.
- You want convenience features such as remote deployment, logging, update rollback, and firmware flashing.
- You want a flexible architecture that can survive hardware changes.

1.2 The artifact stack you should create before buying hardware

Artifact	Purpose	Minimum contents
requirements.yaml	Freezes mission assumptions so hardware can be sized.	Payload, speed, terrain, runtime, autonomy level, arm tasks, safety assumptions.
power_budget.xlsx or power_budget.yaml	Stops under-sizing batteries and DC/DC converters.	Voltage rails, nominal/peak current, duty cycle, fuse size, estimated runtime.
robot_description	Single source of truth for frames and geometry.	URDF/Xacro links, joints, inertial placeholders, sensor frames, collision geometry.
interfaces	Prevents every driver from inventing a custom message.	Topic names, message types, services, actions, QoS policy, ownership.
simulation_worlds	Lets you test navigation and manipulation without the real robot.	Gazebo/Webots/Isaac world, sensor plugins, ros2_control simulation hardware.
bringup	Turns the robot on reproducibly.	Launch files, lifecycle sequence, parameters, udev rules, diagnostics, systemd units.
tests	Links design to validation.	Unit tests, launch tests, bag replay tests, simulation scenarios, hardware smoke tests.

1.3 Immediate working rule

Every hardware candidate should be evaluated by four questions: can I power it, can I physically mount it, can I model it in URDF/SDF, and can I communicate with it using ROS 2 or a clean bridge? If the answer to any of those is unclear, mark the part as research-risk until proven.

2. Define the robot before choosing parts

Requirements are not bureaucracy. They are how you avoid buying a motor, arm, battery, or computer that is impressive in isolation but wrong for the whole machine. Use the tables in this section as your first design worksheet.

2.1 Mission profile questions

Domain	Questions you must answer	Why it matters
Environment	Indoor, outdoor, mixed? Smooth floor, carpet, sidewalk, grass, ramps? Dust, rain, sunlight?	Determines drive type, wheel diameter, LiDAR/camera choice, IP rating, suspension, ground clearance.
Speed	Top speed? Normal cruise speed? Turning radius? Does it operate near people?	Drives motor torque, braking, safety zones, compute latency, Nav2 limits.
Payload	How much mass must the base carry, including battery, arm, sensors, and future expansion?	Drives chassis strength, center of gravity, motor torque, battery energy, wheel loading.
Manipulation	Lift payload? Reach distance? Precision? Two-handed tasks? Mobile manipulation while moving?	Determines arm type, base size, stabilization, MoveIt planning complexity.
Perception	Need obstacle avoidance only, object detection, grasping, people detection, semantic maps, QR/AprilTag localization?	Determines camera type, depth sensing, GPU need, calibration burden.
Autonomy	Teleop only, waypoint navigation, task execution, docking, mapping, manipulation, remote fleet management?	Determines ROS graph complexity, logging, behavior trees, OTA, cloud tools.
Runtime	Minutes or hours? Hot-swappable battery? Charging dock?	Determines battery chemistry, BMS, charger, power rails, weight budget.
Safety	Will it operate near people, pets, property, or expensive equipment?	Determines E-stop, speed limits, bumpers, safety-rated components, watchdog architecture.

2.2 A starter requirements matrix

Fill this matrix before committing to hardware. Replace the example targets with your actual needs.

Requirement ID	Example target	Design consequence	Verification
REQ-MOB-001	Carry total robot mass of 25 kg at 1.0 m/s indoors.	Motor torque, wheel size, battery, frame stiffness.	Timed drive test on smooth floor with payload.
REQ-MOB-002	Climb a 5 degree ramp from rest.	Peak torque and traction margin.	Ramp test with current logging.
REQ-NAV-001	Navigate mapped indoor routes with 0.25 m obstacle clearance.	2D LiDAR, Nav2 costmaps, localization tuning.	Simulation and real hallway route test.
REQ-PER-001	Detect objects within 0.5-3.0 m in front of robot.	RGB-D/stereo camera, GPU or edge accelerator, lighting control.	Bag replay against labeled examples.
REQ-ARM-001	Lift 0.5 kg at 0.4 m reach.	Arm payload/reach selection, base stability, gripper.	Static lift, trajectory, and emergency stop tests.
REQ-PWR-001	Operate 60 minutes at normal duty cycle.	Battery Wh, DC/DC efficiency, thermal management.	Full-system endurance test with logs.
REQ-SAFE-001	Stop drive motion within 0.5 s on E-stop.	Hardware E-stop path, controller watchdog, brake or coast behavior.	Measured stop test on floor and ramp.
REQ-DEV-001	Deploy software remotely with rollback.	Container, systemd, Mender/RAUC, release tags.	Update test from known-good to new version and back.

2.3 Traceability: combine academia's V-model discipline with ROS artifacts

A lightweight V-model fits ROS 2 development well: requirements lead to architecture, architecture leads to robot description and interfaces, those lead to implementation, then tests climb back up from unit tests to integration tests to mission validation. Academic and industrial ROS 2 work repeatedly points to modularity, traceability, simulation, and model-driven practices as ways to manage complex robots [S7, S30, S32].

```
Requirements
-> System architecture
    -> ROS graph, frame tree, power tree, safety tree
        -> URDF/Xacro, ros2_control, firmware interfaces
```

```

-> Node code, MCU firmware, launch files
<- Unit and hardware-interface tests
<- Simulation, bag replay, and subsystem integration tests
<- Mission validation, safety review, field tests

```

The practical version is MBSE-lite: you do not need a heavy SysML tool at the beginning. YAML files, URDF/Xacro, interface definitions, launch files, diagrams, and tests can form a useful system model as long as they are version-controlled and kept in sync.

3. System architecture: the master separation of responsibility

The most important architectural decision is where each control loop lives. Fast, safety-relevant, deterministic loops should live close to hardware. Slower reasoning, planning, perception, and orchestration can live in ROS 2 on Linux.

```

+-----+
| Operator / developer                                     |
|  Foxglove, RViz, CLI, web UI, logs, SSH, OTA, CI/CD    |
+-----+ ROS 2 Linux computer -----+
| Mission executive / behavior tree / task planner       |
| Nav2 navigation | MoveIt 2 manipulation | perception | mapping | logging |
| robot_state_publisher | tf2 | diagnostics | parameter/lifecycle manager |
+-----+ hardware bridge layer -----+
| ros2_control hardware interfaces | micro-ROS agent | CAN/serial bridges |
+-----+ embedded layer -----+
| MCU drive controller | arm safety MCU | power monitor | sensor triggers |
| encoder capture | PID/FOC loops | watchdogs | E-stop response           |
+-----+ physical layer -----+
| motors | motor drivers | encoders | battery | BMS | LiDAR | camera | arm |
+-----+

```

3.1 What belongs on microcontrollers

- Wheel encoder counting and motor-current/velocity/position loops.
- Hard E-stop response and watchdog timeout behavior.
- Power rail measurements, temperature readings, battery state, fan control, and contactor state.
- Gripper timing or force loops when they need deterministic update rates.
- Any action that must remain safe when Linux is busy, crashed, rebooting, or updating.

3.2 What belongs in ROS 2 on Linux

- Sensor drivers and data transport for LiDAR, cameras, IMU, arm, and diagnostics.
- SLAM, localization, path planning, costmaps, behavior trees, and mission orchestration.
- Object detection, pose estimation, depth processing, and semantic perception.
- MoveIt 2 motion planning, collision checking, planning scene, and high-level arm goals.
- Logging, bag recording, remote visualization, parameters, lifecycle management, and software updates.

3.3 Recommended data/control split

Signal	Direction	Rate	Transport	Owner
/cmd_vel	ROS 2 -> base controller	10-50 Hz	ROS topic to hardware interface or MCU bridge	Nav2 / teleop
Wheel encoder ticks	MCU -> ROS 2	50-200 Hz	CAN, serial, micro-ROS, or ros2_control	Drive MCU
Motor PWM/current loop	Inside motor controller/MCU	1-20 kHz	Not a ROS topic	Firmware or motor drive
/scan	LiDAR -> ROS 2	5-20 Hz	Native driver	LiDAR node
/image_raw	Camera -> ROS 2	15-60 fps	Native driver, image_transport	Camera node
/joint_trajectory	Movelt -> arm controller	Goal/action	FollowJointTrajectory action	Movelt/ros2_control
E-stop	Hardware -> power/control	Immediate	Wired circuit, not just software	Safety chain

Design principle: ROS 2 may request motion. It should not be the only thing preventing unsafe motion.

4. ROS 2 distribution, operating system, and compatibility

As of June 25, 2026, ROS 2 Lyrical Luth is the newest LTS release and targets Ubuntu 26.04; ROS 2 Jazzy Jalisco is the previous LTS and targets Ubuntu 24.04. Lyrical gives the longest runway, while Jazzy may have broader hardware-vendor support because many vendors have already tested it [S1, S2, S3].

4.1 Distro decision table

Choice	When to choose it	Caution
ROS 2 Lyrical + Ubuntu 26.04	Best for a project starting from scratch if the drivers you need already support it. Longest LTS horizon through May 2031 [S1].	It is very new. Some third-party drivers, tutorials, and binary packages may lag.
ROS 2 Jazzy + Ubuntu 24.04	Best near-term default if you want proven package availability, Gazebo Harmonic pairing, and broad vendor testing. Supported through May 2029 [S2].	Shorter support runway than Lyrical; eventual migration needed.
ROS 2 Humble + Ubuntu 22.04	Use only if a critical vendor stack requires it today.	Older baseline; avoid for a new blank-slate design unless forced.
ROS 2 Rolling	Use for experimentation with newest APIs or contributing upstream.	Not a stable product baseline; changes can break you.

4.2 My recommended starting baseline

Create the design repository so it can build on both Jazzy and Lyrical, but start physical hardware integration on one stable distro. For a hardware-undecided robot in mid-2026, choose one of these two paths:

Path	Baseline	Why
Conservative hardware path	Ubuntu 24.04 + ROS 2 Jazzy + Gazebo Harmonic	More vendor drivers and tutorials are likely to have been tested. Good for first physical prototype.
Long-horizon software path	Ubuntu 26.04 + ROS 2 Lyrical + Gazebo Jetty	New LTS runway. Good if all critical drivers are available or you are comfortable building from source.

4.3 Mac and Windows development compatibility

Treat Mac and Windows as development clients, not as the primary onboard runtime. The robot should run Linux because ROS 2, drivers, real-time tuning, Gazebo integration, GPU stacks, udev, systemd, and hardware access are most natural there. Official ROS 2 documentation provides Windows binary installation and macOS source-build paths, but those are not the simplest way to debug custom robot hardware [S17, S18].

Host	Good workflow	Avoid relying on
Windows	VS Code Remote-SSH into robot; WSL2 for ROS CLI experiments; Docker Desktop/dev containers; Foxglove in browser.	Native Windows as only robot dev environment for hardware drivers.
macOS Intel/Apple Silicon	Remote-SSH to Ubuntu robot or Linux dev box; Docker for non-hardware packages; Foxglove/browser UI.	Native source-built ROS as the main integration path, unless you enjoy toolchain work.
Linux laptop/desktop	Best full-stack dev setup: native ROS, Gazebo, RViz, USB devices, network sniffing, flashing tools.	Running heavy simulation on the robot while expecting reliable autonomy.

4.4 Recommended developer environment

Development machine:

- VS Code + Remote SSH extension
- Git + pre-commit + clang-format/black
- Docker Desktop or native Docker for dev containers
- Foxglove desktop/browser for visualization

Robot computer:

- Ubuntu Linux
- ROS 2 LTS selected above
- systemd services for bringup
- udev rules for stable device names
- Docker/Compose only if GPU, networking, and real-time constraints are understood

Repository:

```
robot_ws/
src/
  my_robot_description/
  my_robot_bringup/
  my_robot_base/
  my_robot_navigation/
  my_robot_manipulation/
  my_robot_perception/
  my_robot_firmware_interfaces/
  my_robot_tests/
```

5. ROS 2 concepts you should design around now

ROS 2's value is not only package reuse. It gives you a common vocabulary: nodes, topics, services, actions, parameters, lifecycle nodes, QoS, tf2 transforms, bag files, and launch files. These abstractions let your robot stay modular when hardware changes. ROS 2 documentation describes topics as continuous data streams, services as short request-response interactions, and actions as long-running goals with feedback [S4].

5.1 Interface selection

ROS 2 interface	Use for	Robot examples
Topic	Continuous data streams where subscribers consume latest data.	/scan, /image_raw, /joint_states, /odom, /tf, /cmd_vel
Service	Short command/response interactions that complete quickly.	Reset odometry, set mode, request calibration state, clear costmap.
Action	Long-running task with feedback, cancellation, and result.	NavigateToPose, FollowJointTrajectory, docking, mapping mission.
Parameter	Configuration loaded at launch or changed deliberately.	Max speed, topic names, PID gains, sensor frame names.
Lifecycle	Nodes that need configure/activate/deactivate transitions.	LiDAR driver, localization, Nav2 servers, hardware bridge.

5.2 QoS policy defaults for your robot

ROS 2 QoS controls reliability, durability, history, queue depth, deadline, liveliness, and lifespan. Connections only form when publisher and subscriber QoS policies are compatible [S5].

Data type	Suggested QoS	Reason
Camera images	SensorDataQoS / best effort / small queue	Dropping old frames is often better than building latency.
LiDAR scans	SensorDataQoS / best effort or reliable depending on driver	Navigation wants recent data; do not queue stale scans.
/cmd_vel	Reliable, small queue	Commands should arrive, but stale commands must timeout in firmware.
/tf and /tf_static	Use standard tf2 defaults	Transform tools expect conventional behavior.
Diagnostics	Reliable, moderate queue	Status should not be silently lost.
Map	Transient local/durable where appropriate	Late-joining nodes may need the map.

5.3 tf2 frame tree: define this before CAD is done

Frame naming becomes a design contract. A clean frame tree lets you swap sensors, simulate, calibrate, visualize, localize, and plan without rewriting code.

```
map
-> odom
  -> base_footprint
    -> base_link
      -> base_laser
      -> front_camera_link
        -> front_camera_optical_frame
      -> imu_link
      -> arm_base_link
        -> arm_link_1 -> ... -> tool0
```

Frame	Meaning	Owner
map	Global map frame, discontinuous corrections allowed.	Localization / SLAM
odom	Locally smooth odometry frame, drifts over time.	Odometry estimator
base_footprint	2D projection of robot base on ground.	Robot description / localization
base_link	Main body reference frame, centerline near chassis center.	URDF
sensor frames	Physical mounting points for LiDAR, camera, IMU.	URDF plus calibration
arm frames	Arm kinematic chain from arm_base_link to tool0.	Arm description / MoveIt

5.4 ros2_control and micro-ROS in the architecture

ros2_control is the natural abstraction for robot hardware interfaces and controllers. It provides a framework for integrating hardware, controllers such as joint trajectory and differential drive controllers, real-time tools, and simulator integrations [S10]. micro-ROS brings ROS 2 concepts to microcontrollers using an MCU-optimized client API and an agent that connects MCU nodes into the ROS 2 graph [S11].

Approach	Best use	Tradeoff
ros2_control hardware interface on Linux	Motor drives or arm controllers with clean Linux-accessible APIs.	Great integration with controllers; Linux is still not your hard safety layer.
micro-ROS on MCU	MCU participates as ROS-like node with publish/subscribe/services.	Powerful, but adds embedded complexity and memory constraints.
Custom serial/CAN protocol + ROS bridge	Simple drive base, robust firmware, deterministic bus behavior.	You maintain protocol and bridge, but can keep firmware small.
Direct vendor ROS driver	Off-the-shelf arm, LiDAR, camera, smart motor controller.	Fastest start; verify distro support and long-term maintenance.

6. Mechanical platform and drive-base selection

Choose the drive base from the environment and autonomy requirement, not from what looks cool. For ROS 2 navigation, differential drive is the fastest route to a successful autonomous prototype. Holonomic and Ackermann robots are valid, but they increase planning, control, and mechanical complexity.

6.1 Drive-base decision matrix

Base type	Advantages	Disadvantages	Good first prototype?
Differential drive	Simple mechanics, excellent ROS/Nav2 support, turns in place, easy odometry.	Can scrub tires; traction depends on caster/support design.	Yes. Best default.
Skid steer / tracks	Rugged, outdoor traction, can turn in place.	Odometry is worse due to slip; more power and surface damage.	Maybe, if terrain requires it.
Mecanum / omni	Holonomic motion, excellent for tight indoor spaces.	Lower efficiency, wheel sensitivity, harder control and localization.	Not for first autonomous build unless holonomic is required.
Ackermann	Efficient at speed, car-like outdoor behavior.	Cannot turn in place; planners/controllers need car-like constraints.	Good for vehicle-like robot, not general indoor mobile manipulator.
Legged	Terrain capability.	Extremely complex mechanics/control.	No, unless legged locomotion is the project.

6.2 Default mechanical recommendation

Start with a four-wheel differential or two-wheel differential plus stable caster design. Put the battery low and central. Put the LiDAR high enough to see chair/table legs but not so high that it misses low obstacles. Mount the main compute where airflow and service access are good. Reserve a flat, reinforced arm mount near the base centerline, and do not put a tall arm on a narrow chassis without a stability calculation.

6.3 Torque and speed sizing formulas

Motor sizing starts from force at the ground. Use conservative estimates and then validate with current logs.

Inputs:

- m = total robot mass including payload [kg]
- g = 9.81 [m/s²]
- r = wheel radius [m]

```

a      = desired acceleration [m/s^2]
theta  = ramp angle [radians]
Crr    = rolling resistance coefficient
n      = number of driven wheels
eta    = drivetrain efficiency, e.g. 0.70-0.85
SF     = safety factor, e.g. 1.5-2.5

```

Approximate required tractive force:

$$F = m \cdot a + m \cdot g \cdot \sin(\theta) + Crr \cdot m \cdot g \cdot \cos(\theta)$$

Torque per driven wheel:

$$T_{\text{wheel}} = (F \cdot r / n) \cdot SF / \eta$$

Wheel speed for target linear speed v :

$$\text{wheel_rpm} = (v / (2 \cdot \pi \cdot r)) \cdot 60$$

Do not size only for flat-floor cruising. Size for starting from rest, turning, ramp climbing, payload, and wheel scrub. If you later add an arm, camera mast, or larger battery, the center of gravity and motor load change.

6.4 Chassis and mounting plan

Mounting zone	Recommended practice	Avoid
Battery	Low, central, mechanically retained, protected from puncture, easy to remove.	High-mounted battery or battery held only by tape/Velcro.
Main compute	Accessible, ventilated, isolated from motor EMI, protected from dust/debris.	Buried under arm mount with no airflow.
LiDAR	Rigid horizontal mount with clear 360/270 degree view; known frame transform.	Flexible mast that vibrates or sees robot body.
Camera	Rigid front mount with known pitch; avoid vibration; plan calibration access.	Mounting behind reflective plastic or near high-vibration motors.
Arm	Reinforced plate tied into chassis; centerline if possible; check tipping.	Mounting on thin sheet metal or at extreme edge without outriggers.
E-stop	Reachable from outside, hardware interrupting motion power or enable line.	Software-only button in a GUI.

7. Motors, motor drivers, encoders, and low-level motion

For drive wheels, the normal choices are brushed DC gearmotors with encoders, BLDC motors with encoders/Hall sensors, or integrated servo wheel modules. The best first robot is usually not the most exotic motor; it is the motor you can power, mount, measure, command, and stop safely.

7.1 Motor type comparison

Motor type	Pros	Cons	Where it fits
Brushed DC gearmotor + encoder	Simple, affordable, easy velocity control, many drivers.	Brush wear, lower efficiency, electrical noise.	First differential-drive prototypes.
BLDC + encoder/Hall + FOC controller	Efficient, powerful, smoother, good for larger robots.	More tuning, more expensive, controller complexity.	Heavy base, high runtime, higher speed.
Stepper	Simple position control at low speed.	Can miss steps, inefficient, poor for wheel traction dynamics.	Mechanisms, not primary drive wheels.
Smart servo / integrated module	Integrated control and feedback, clean APIs.	Vendor lock-in, cost, current limits.	Arms, grippers, compact joints.

7.2 Drive controller architecture

```

ROS 2 /cmd_vel
-> base_controller node or ros2_control diff_drive_controller
  -> velocity setpoints per wheel
    -> MCU or motor-drive command
      -> encoder/current/PID/FOC loop
        -> motor phases/PWM

```

```

Feedback path:
encoder ticks + motor current + driver status
-> odometry estimator / diagnostics / safety monitor

```

Keep a firmware watchdog: if no valid command arrives within a short timeout, the drive controller should command zero velocity or enter a configured safe stop. Never let a stale /cmd_vel continue moving the robot.

7.3 Encoder recommendations

- Use quadrature encoders on each driven wheel at minimum. Absolute encoders are useful but not required for a first base.
- Measure encoder ticks in the MCU, not in a non-real-time Linux loop.
- Timestamp feedback near acquisition, or include enough timing information to compute velocities correctly.
- For high-slip bases, add IMU fusion and be honest about odometry quality.
- Bench-test encoders with wheels off the ground before installing the full chassis.

7.4 Motor driver selection criteria

Criterion	What to check
Voltage/current	Continuous and peak current at your battery voltage, with thermal headroom.
Control mode	Velocity/current/position modes; command interface; fault reporting.
Feedback	Encoder/Hall support, current sense, temperature, fault codes.
Interface	CAN, UART, USB, PWM, RS485; deterministic enough for your control layer.
Regeneration	What happens during braking? Can the battery/BMS absorb current?
Safety	Enable pin, brake input, fault line, watchdog, current limits, emergency stop behavior.
ROS support	Existing driver or clean protocol to bridge into ROS 2.

8. Power system and battery design

Robots fail in boring ways: brownouts, insufficient peak current, loose connectors, overloaded DC/DC converters, overheating, EMI from motors, and batteries that cannot absorb regenerative braking. Treat the power tree as a first-class design artifact.

8.1 Power tree template

```

Battery pack + BMS
-> main fuse / breaker
-> contactor or main power switch
-> E-stop controlled motor-enable path
-> high-power rail: motor drivers / arm drives
-> DC/DC 12 V rail: sensors, fans, relays
-> DC/DC 5 V rail: SBC peripherals, USB hubs, logic
-> isolated or filtered rail where needed: cameras / analog sensors

```

```

Telemetry:
battery voltage, battery current, rail voltage, rail current,
BMS faults, temperatures, E-stop state, charger/dock state

```

8.2 Battery sizing formula

Estimate energy:

$$\text{Wh}_{\text{required}} = \text{average_power_W} * \text{runtime_hours} / \text{usable_fraction} / \text{conversion_efficiency}$$

Example:

```
average robot power = 180 W
runtime target = 1.0 hour
usable fraction = 0.80
conversion efficiency = 0.90
Wh_required = 180 * 1.0 / 0.80 / 0.90 = 250 Wh
```

Then check peak current:

$$\text{peak_current_A} = \text{peak_power_W} / \text{pack_voltage_V}$$

8.3 Battery chemistry choice

Chemistry	Pros	Cons	Notes
Li-ion	High energy density, common packs.	Needs good BMS, thermal care, fire risk if abused.	Good for compact runtime.
LiFePO4	Safer chemistry, long cycle life, robust.	Lower energy density, heavier.	Excellent for larger ground robots.
Lead-acid	Cheap, tolerant, easy charging.	Very heavy, poor energy density.	Only for crude test platforms.
Bench supply	Great for early electronics tests.	Not mobile; current-limited behavior can hide issues.	Use before battery integration.

8.4 Power design checklist

- Fuse every branch close to the source. Size wires by current, length, voltage drop, and temperature.
- Use locking connectors. Label voltage rails. Prevent plugging a sensor into the wrong voltage.
- Separate dirty motor power from sensitive camera/compute power with layout, filtering, and grounding discipline.
- Make E-stop remove motor enable or motor power through hardware. Linux can also know E-stop state, but it should not be the only stop mechanism.
- Log voltage sag under acceleration. Brownouts during motor startup usually mean wiring, battery, BMS, or DC/DC margins are wrong.
- Plan charging and service access before final chassis design.

9. Low-level hardware model and microcontroller architecture

Your robot should have at least one dedicated embedded controller for motor control and safety. Larger robots often split embedded responsibilities: drive MCU, power MCU, arm/gripper MCU, and sensor timing MCU. The split should follow safety and wiring boundaries, not arbitrary software preference.

9.1 Minimum MCU responsibilities

Function	Minimum implementation	ROS-visible output
Drive control	Read encoders, command motor drivers, run velocity loop or pass setpoints to smart drives.	/odom or wheel states; diagnostics
Command watchdog	Stop or disable drive when command stream times out.	Watchdog status diagnostic
E-stop input	Immediate inhibit of motor enable or drive command.	E-stop state topic/diagnostic
Power telemetry	Measure battery/rail voltage and current; read BMS if available.	/battery_state, diagnostics
Fault handling	Latch critical faults until reset policy is met.	Fault code service/topic

9.2 MCU and bus selection

Option	Use when	Notes
STM32 / Nucleo / custom STM32	You want deterministic embedded control and broad peripherals.	Good for motor/encoder/CAN work; requires embedded toolchain comfort.
Teensy 4.x	Fast prototyping with high performance and Arduino-like workflow.	Excellent for hobby/prototype; consider long-term supply and robustness.
ESP32	You want Wi-Fi/BLE, OTA, simple sensor nodes, or low-cost peripherals.	Great convenience; less ideal for critical drive loops unless carefully designed.
RP2040/RP2350 class	Low-cost deterministic peripheral controller.	Useful for small I/O roles; check motor-control requirements.
CAN / CAN-FD	Multiple controllers, motor drives, noisy robot wiring.	Excellent robot bus; robust and addressable.
RS485	Simple multidrop serial in noisy environments.	Good for custom protocols; no built-in arbitration like CAN.
USB serial	Prototype bench and simple single MCU.	Convenient but cable/device naming and EMI can be annoying.
I2C/SPI	Short-board connections.	Do not run across a robot chassis unless specifically engineered.

9.3 micro-ROS versus custom protocol

micro-ROS is attractive because it brings ROS concepts to the MCU and can integrate with regular ROS tools through the micro-ROS agent [S11]. For a first drive base, a custom binary protocol over CAN or serial can be simpler and more deterministic. The decision should be based on firmware complexity, memory limits, update rate, debugging strategy, and whether you need MCU nodes to appear in the ROS graph.

Question	Choose micro-ROS if...	Choose custom bridge if...
Need native ROS graph on MCU?	Yes, you want MCU publishers/services visible as ROS-like nodes.	No, the Linux bridge can translate messages.
Firmware size/complexity	You can manage RTOS, memory pools, agent setup, and ROS message generation.	You want small, auditable firmware with a tight protocol.
Transport	Serial/UDP/Wi-Fi/Bluetooth support fits your architecture [S11].	CAN/CAN-FD arbitration and robust motor-network behavior are primary.
Real-time loop	The ROS-facing layer is not the inner control loop.	The protocol only carries setpoints/status; inner loops are isolated.

9.4 Firmware state machine

```

BOOT
-> SELF_TEST
  -> SAFE_IDLE      (motors disabled, sensors reporting)
  -> ARMED         (motion allowed, watchdog active)
    -> FAULT       (latched or recoverable fault)
    -> UPDATE      (firmware update mode, motion disabled)
  -> FAULT

```

Critical invariant:

Any transition caused by E-stop, watchdog timeout, overcurrent, encoder fault, thermal fault, or BMS fault must end in a safe state.

9.5 Firmware update and OTA strategy

Do firmware updates in layers. First use a wired bootloader/debugger during bench development. Add OTA only after you have a reliable recovery path. ESP-IDF supports OTA partition workflows for Espressif devices [S28], PlatformIO Remote can upload to remote devices in some workflows [S27], and Linux-side OTA can be handled by mature embedded update frameworks such as Mender or RAUC [S25, S26].

Layer	Recommended update method	Rollback/recovery requirement
MCU bootloader	SWD/JTAG/USB DFU first; OTA only after bench validation.	Physical recovery header always accessible.
Robot application packages	Debian packages, Docker images, or colcon build from tagged release.	Keep previous release installed or container image cached.
Robot OS image	Mender/RAUC A/B update or immutable image strategy.	A/B rootfs or verified rollback; do not brick robot remotely.
Parameters/maps	Versioned config bundle, not hand-edited on robot.	Snapshot known-good configs before field changes.

10. Main compute architecture

Main compute depends on perception ambition. Navigation with 2D LiDAR can run on modest hardware. Depth cameras, point clouds, neural networks, and arm planning can quickly dominate CPU/GPU, memory bandwidth, thermals, and storage.

10.1 Compute options

Option	Strengths	Weaknesses	Best use
Raspberry Pi 5	Low cost, small, good community, 2.4 GHz quad-core Arm Cortex-A76 and RAM variants up to 16 GB [S20].	Limited GPU/AI acceleration for heavy perception; USB bandwidth/power planning matter.	Light navigation prototype, sensor bridge, education.
NVIDIA Jetson Orin Nano Super	Compact edge AI platform; NVIDIA lists up to 67 TOPS AI performance for the Super developer kit [S19].	JetPack/CUDA/ROS distro compatibility must be managed; thermals/power matter.	Vision AI plus ROS 2 on mobile robot.
x86 mini PC	Strong CPU, NVMe storage, easy Ubuntu, often best desktop-like ROS experience.	Higher power; GPU may be absent unless selected.	Robust Nav2/MoveIt/perception development, simulation-adjacent testing.
x86 + discrete GPU	Best for heavy perception, training-adjacent workloads, dense mapping.	Power, size, heat, cost.	Large robot or lab platform.

10.2 Default compute recommendation

For a serious autonomous mobile manipulator prototype, I would choose either an x86 mini PC with NVMe storage or a Jetson Orin-class module/dev kit. Use Raspberry Pi 5 only if the first phase is lightweight navigation and you are comfortable offloading AI or reducing sensor load.

10.3 Onboard compute checklist

- NVMe storage if you plan to record bags with images or point clouds.
- Reliable 12-20 V or USB-C power input with margin; avoid marginal phone chargers.
- Dedicated cooling. Robots collect dust and operate in enclosures.
- Ethernet for LiDAR and reliable development access; Wi-Fi for convenience only.

- Enough USB bandwidth and stable device naming for cameras, MCUs, and debug tools.
- A real-time kernel is optional early; good architecture and MCU control matter more.

10.4 GPU and AI acceleration

If your robot only needs 2D SLAM and waypoint navigation, a GPU is not the first bottleneck. If it needs object detection, segmentation, visual servoing, VLMs, or dense 3D perception, plan GPU acceleration early. NVIDIA Isaac ROS provides CUDA-accelerated ROS 2 packages for autonomous robots on Jetson and other NVIDIA platforms [S36].

11. Sensors and mounting strategy

Sensor selection should match the first autonomy milestone. A practical first autonomy stack is wheel encoders + IMU + 2D LiDAR for navigation, plus one front RGB-D or stereo camera for perception and manipulation. Add 3D LiDAR only when the problem requires 3D mapping or high-speed outdoor perception.

11.1 LiDAR choices

LiDAR type	Use case	Pros	Cons
2D spinning LiDAR	Indoor mapping, Nav2, obstacle detection.	Simpler data, mature Nav2 workflow, lower compute.	Sees only one plane; misses obstacles above/below scan.
3D LiDAR	Outdoor, uneven environments, 3D mapping, complex obstacles.	Rich geometry and long-range perception.	High cost, high bandwidth, point-cloud compute.
Depth camera as depth sensor	Short-range obstacle and manipulation perception.	Gives RGB + depth; useful for grasping.	Sunlight/reflective surfaces/range limitations depending technology.

Driver availability matters. Examples of actively documented ROS 2 driver paths include Slamtec slidar_ros2, Hokuyo urg_node2, SICK sick_scan_xd, and Ouster ROS drivers [S23].

11.2 Camera choices

Camera type	Best for	Notes
USB RGB camera	Simple teleop, image capture, object detection in good lighting.	Low cost; depth requires another sensor.
RGB-D camera	Indoor perception, obstacle detection, grasping, object distance.	Good first manipulation camera. Check ROS 2 driver and lighting/range limits.
Stereo camera	Depth without active projection; outdoor variants exist.	Needs texture and calibration; compute cost varies.
Global-shutter camera	Fast motion, visual odometry, fiducials on moving robot.	Often more expensive but avoids rolling-shutter distortion.
OAK/DepthAI camera	On-device vision pipelines and ROS 2 components.	Luxonis documents ROS 2 drivers and parameterized components [S21].
Orbbec depth camera	RGB-D pipeline with ROS 2 SDK wrapper.	Orbbec provides ROS 2 SDK documentation and advanced features [S22].

11.3 IMU, wheel odometry, and sensor fusion

Wheel encoders provide local motion estimates but drift. IMUs help observe angular velocity and short-term acceleration. For many indoor bases, use robot_localization or an equivalent EKF to fuse wheel odometry and IMU, then use SLAM/AMCL/Nav2 for map-level localization. Nav2 documentation includes guidance for smoothing odometry with robot_localization [S8].

11.4 Mounting and calibration rules

- Rigid mounts beat clever software compensation. If the camera or LiDAR moves, the frame tree is wrong.
- Keep sensor frames measurable. Record x/y/z and roll/pitch/yaw from base_link.
- Give LiDAR an unobstructed field of view; model any robot body occlusions in simulation.
- Pitch cameras deliberately. A front camera for navigation/perception is usually angled to see the near floor and task space, not the ceiling.
- Calibrate camera intrinsics, camera-to-base extrinsics, and camera-to-arm/hand-eye if grasping.
- Record bags of every new sensor before writing algorithms. First validate data rate, timestamps, frame IDs, and QoS.

12. Robot arm strategy

A mobile manipulator is two robots bolted together: a mobile base and a manipulator. The integration difficulty comes from stability, coordinate frames, collision geometry, power, control timing, and calibration. If manipulation is central, buy a ROS 2-supported arm first; build a custom arm only after you understand the task and payload.

12.1 Arm category decision table

Arm category	Pros	Cons	Recommendation
Off-the-shelf 6-DoF arm	Known kinematics, MoveIt configs often available, lower software risk.	Cost, payload/reach fixed, integration still required.	Best if manipulation is a real project goal.
Educational servo arm	Affordable, easy learning platform.	Low payload, compliance/backlash, limited precision.	Good for learning, not robust mobile manipulation.
Dynamixel/custom arm	Flexible, ROS ecosystem, good educational/modular path.	Mechanical design and calibration burden are yours.	Good if custom arm design is the research goal.
Industrial/collaborative arm	Payload, repeatability, safety features, vendor support.	Cost, weight, power, safety requirements.	Good for serious manipulation if budget allows.

ROS 2 driver examples exist for xArm, myCobot, Kinova Kortex, and Dynamixel ecosystems [S24]. Treat driver support as a purchase requirement, not a bonus.

12.2 Arm mounting on a mobile base

- Mount as close to the center of the base as the task allows.
- Check tipping in worst-case reach: arm extended, payload held, base braking, slope, and battery placement.
- Add collision geometry for the chassis, sensor mast, arm base, gripper, and payload.
- Route cables through strain relief and keep them out of wheel/arm sweep zones.
- Provide a mechanical hard stop or protected cable loop where possible.
- If lifting significant payloads, consider outriggers, a wider base, or lower speed limits during arm motion.

12.3 MoveIt 2 architecture

MoveIt 2 is the major ROS 2 manipulation framework, covering motion planning, kinematics, manipulation, 3D perception, control, and navigation integration [S9]. A proper MoveIt integration starts with a clean URDF/Xacro, joint limits, collision geometry, semantic robot description (SRDF), controller mapping, and planning scene.

```

MoveIt 2 high-level flow:
task request
-> planning scene update
-> inverse kinematics / constraints
-> collision checking
-> trajectory planning
-> time parameterization
-> FollowJointTrajectory action
-> arm controller / ros2_control / vendor driver

```

12.4 Mobile manipulation rule

Do not try full mobile manipulation on day one. First make the base navigate. Then make the arm move safely while the base is stationary. Then add perception and grasping. Only after those are stable should you coordinate base and arm motion.

13. ROS 2 workspace, package layout, and naming conventions

A clean package structure is a design tool. It keeps simulation, real hardware, perception, navigation, manipulation, and firmware interfaces separable. It also makes it easier to test changes without booting the entire robot.

13.1 Recommended repository layout

```

my_robot_ws/
src/
  my_robot_description/      # URDF/Xacro, meshes, frames, inertials
  my_robot_bringup/         # top-level launch, lifecycle, system params
  my_robot_simulation/      # Gazebo/Webots/Isaac worlds and sim launch
  my_robot_base/            # base driver or ros2_control hardware plugin
  my_robot_firmware_interfaces/ # messages, protocol docs, MCU schemas
  my_robot_navigation/      # Nav2 params, maps, localization configs
  my_robot_manipulation/    # MoveIt configs, gripper, arm launch
  my_robot_perception/      # camera, LiDAR processing, detection nodes
  my_robot_diagnostics/     # health aggregation, monitoring, status
  my_robot_tests/           # launch tests, bag tests, sim scenarios
tools/
  flash_mcu.sh
  record_bag_profiles.sh
  make_release_bundle.sh
docs/
  requirements.yaml
  power_budget.yaml
  frame_tree.md
  interface_contract.md

```

13.2 Naming conventions

Item	Convention	Example
Packages	Lowercase with robot prefix.	my_robot_navigation
Frames	Lowercase, descriptive, consistent with REP practices.	base_link, base_laser, front_camera_optical_frame
Topics	Semantic, not vendor-specific where possible.	/scan rather than /rplidar_scan unless multiple LiDARs
Parameters	Keep per-node YAML files with comments.	nav2_params.yaml, base_controller.yaml
Launch files	One launch file per integration level.	sim.launch.py, base.launch.py, robot.launch.py
Branches	Stable main; hardware experiment branches short-lived.	feature/lidar_mount_v2

13.3 Interface contract template

```
# docs/interface_contract.md

Topic: /cmd_vel
  Type: geometry_msgs/msg/Twist
  Publisher: nav2_controller or teleop
  Subscriber: base_controller
  QoS: reliable, depth 1-5
  Safety: firmware stops if no command within 200 ms

Topic: /odom
  Type: nav_msgs/msg/Odometry
  Publisher: base_controller or EKF
  Frame: odom -> base_link/base_footprint
  Rate: 30-100 Hz

Action: /navigate_to_pose
  Type: nav2_msgs/action/NavigateToPose
  Owner: Nav2 BT Navigator

Action: /arm_controller/follow_joint_trajectory
  Type: control_msgs/action/FollowJointTrajectory
  Owner: arm controller or vendor driver
```

14. ROS 2 graph for the first complete robot

This section gives a concrete ROS graph that can evolve from simulation to hardware. You can remove pieces at first, but the interfaces should remain stable.

14.1 Core node list

Node	Package/source	Responsibilities
robot_state_publisher	robot_state_publisher	Publishes static and joint-derived transforms from URDF.
joint_state_broadcaster	ros2_control	Publishes joint states from hardware/sim.
base_controller	custom or ros2_control	Converts /cmd_vel to wheel commands and publishes odometry/diagnostics.
micro_ros_agent or can_bridge	micro-ROS/custom	Connects MCU network to ROS graph.
lidar_driver	vendor driver	Publishes LaserScan or PointCloud2.
camera_driver	vendor driver	Publishes images, camera info, depth/point cloud.
ekf_node	robot_localization	Fuses wheel odometry and IMU.
slam_toolbox or AMCL	mapping/localization	Builds map or localizes in known map.
Nav2 servers	nav2	Planner, controller, costmaps, behavior tree navigator, recoveries.
move_group	Movelit 2	Motion planning and planning scene.
perception nodes	custom/AI packages	Detection, segmentation, tracking, grasp candidates.
diagnostic aggregator	diagnostics	Aggregates status for operator UI and logs.
foxglove_bridge	Foxglove	Remote visualization and observability [S15].

14.2 Startup sequence

1. Boot Linux and start core systemd services.
2. Start MCU bridge/micro-ROS agent with motors disabled.
3. Start robot_state_publisher and static transforms.
4. Start sensor drivers in inactive/configured lifecycle state if supported.
5. Start base controller and diagnostics.
6. Start localization/SLAM.
7. Start Nav2 lifecycle manager and activate Nav2 stack.
8. Start MoveIt and arm controller only after arm self-test passes.
9. Enable motion only after E-stop clear, battery healthy, and transforms valid.

14.3 Lifecycle and fault state

Use lifecycle where a node needs controlled configure/activate/deactivate transitions. A LiDAR can be configured but inactive until the robot is armed. Nav2 already uses lifecycle managers. Your custom base controller should expose health state and fail safe if transforms, odometry, or MCU communication are invalid.

15. Modeling the robot: URDF, Xacro, SDF, and ros2_control

The robot description is a core design artifact, not an afterthought for visualization. It defines coordinate frames, joint limits, collision geometry, inertial properties, sensor poses, and control interfaces. It also becomes the bridge between CAD, RViz, Gazebo, MoveIt, and hardware.

15.1 URDF/Xacro modeling workflow

Step	Output	Validation
Define base frames	base_link, base_footprint, wheel links, sensor links.	View in RViz; check axes.
Add visual geometry	Meshes or simple shapes.	Looks like the robot.
Add collision geometry	Simplified boxes/cylinders for body, wheels, arm.	MoveIt/Gazebo collision checks work quickly.
Add inertials	Mass and inertia estimates.	Gazebo does not behave wildly.
Add joints	Continuous/fixed/revolute joints and limits.	joint_state_publisher and robot_state_publisher work.
Add ros2_control tags	Hardware interfaces and controller mappings.	Controller manager loads in sim/hardware.
Add sensors	LiDAR/camera/IMU frames and plugins.	Simulated sensor topics match real topic contracts.

15.2 CAD-to-URDF rule

Do not export high-detail CAD as collision geometry. Use simple primitive collision shapes and decimated visual meshes. A visually pretty model that makes collision checking slow is a bad robot model.

15.3 Minimal URDF/Xacro fragment

```
<robot xmlns:xacro="http://www.ros.org/wiki/xacro" name="my_robot">
  <link name="base_link">
    <visual>
      <geometry><box size="0.55 0.42 0.18"/></geometry>
    </visual>
    <collision>
      <geometry><box size="0.55 0.42 0.18"/></geometry>
    </collision>
    <inertial>
      <mass value="12.0"/>
      <!-- Replace placeholder inertia with calculated values. -->
      <inertia ixx="0.20" ixy="0" ixz="0" iyy="0.30" iyz="0" izz="0.40"/>
    </inertial>
  </link>

  <link name="base_laser"/>
  <joint name="base_laser_joint" type="fixed">
    <parent link="base_link"/>
    <child link="base_laser"/>
    <origin xyz="0.18 0 0.24" rpy="0 0 0"/>
  </joint>
</robot>
```

15.4 ros2_control simulation and hardware swap

The goal is to keep controllers and higher-level nodes unchanged when moving between simulation and real hardware. In simulation, `ros2_control` talks to a simulated hardware interface. On the real robot, it talks to your motor drivers/MCU bridge. This pattern reduces rewrite risk.

```
Same controller config:
  diff_drive_controller
  joint_state_broadcaster
  arm_controller

Simulation hardware:
  gz_ros2_control or simulator-specific hardware interface

Real hardware:
  my_robot_base_hardware plugin
  vendor arm driver or custom hardware interface
```

16. Simulation and digital twin workflow

Simulation lets you work while hardware is undecided. It will not perfectly predict reality, but it can validate interfaces, frames, launch files, Nav2 behavior, MoveIt collision geometry, and many failure cases. Gazebo provides official ROS integration through `ros_gz` and recommended ROS/Gazebo pairings, with Jazzy paired to Gazebo Harmonic and Lyrical paired to Gazebo Jetty [S12].

16.1 Simulator choice

Simulator	Best for	Notes
Gazebo	Default ROS 2 robot simulation, sensors, <code>ros2_control</code> , Nav2 worlds.	Use the recommended Gazebo release for your ROS distro [S12].
Isaac Sim	High-fidelity visual simulation, synthetic data, NVIDIA GPU workflows.	NVIDIA documents a ROS 2 bridge and recommends Humble/Jazzy in current Isaac Sim docs [S34].
Webots	Education, quick robot simulation, ROS 2 integration tutorials.	Official ROS 2 tutorials exist for Webots [S35].
MuJoCo	Dynamics-heavy arms/legged/control research.	<code>mujoco_ros2_control</code> can expose MuJoCo through <code>ros2_control</code> interfaces [S37].

16.2 Simulation levels

Level	Purpose	Example test
Interface simulation	Nodes start and topics connect with fake data.	Launch robot without hardware; verify <code>/tf</code> , <code>/joint_states</code> , <code>/diagnostics</code> .
Physics simulation	Test base motion, sensors, collisions, Nav2 in a world.	Navigate a hallway loop in Gazebo.
Perception simulation	Generate camera/LiDAR data for algorithms.	Detect target objects or obstacles in simulated scenarios.
Hardware-in-the-loop	Real MCU/motor driver with simulated robot or constrained bench.	Send <code>/cmd_vel</code> through real drive controller with wheels lifted.
Bag replay	Re-run real data through algorithms without robot motion.	Replay a hallway LiDAR/camera bag after changing perception code.

16.3 Digital twin development sequence

Week 1-2:

Create URDF/Xacro with `base_link`, `wheels`, `LiDAR`, `camera`, `arm placeholder`.
Launch `robot_state_publisher` and view in RViz.

Week 3-4:

Add Gazebo world and differential drive simulation.
Publish `/scan`, `/odom`, `/tf`, `/joint_states`.

Week 5-6:

Bring up Nav2 in simulation with a simple map.
Record bags and tune costmaps/controllers.

Week 7-8:

Add arm model and MoveIt configuration.
Test collision geometry and simple pick/place motions.

Before hardware purchase:

Use the model to validate sensor placement, frame names, chassis size, rough stability, service access, and software interfaces.

16.4 What simulation cannot guarantee

- Motor torque margins under real carpet, ramps, tire scrub, or wheel slip.
- Camera behavior under real lighting, glare, motion blur, and vibration.
- Battery voltage sag, connector heating, EMI, and BMS behavior.
- Arm flex, gripper compliance, cable snags, and payload shifts.
- Safety performance unless the real safety chain is tested.

17. Navigation stack: odometry, SLAM, localization, and Nav2

Nav2 is the ROS 2 navigation framework and successor to the ROS Navigation Stack. Its documentation describes perception, planning, control, localization, visualization, dynamic obstacle avoidance, and behavior-tree based navigation [S8]. Start with a 2D LiDAR navigation stack before attempting complex 3D semantic navigation.

17.1 Navigation architecture

```

Wheel encoders + IMU
-> base odometry / EKF
    -> odom -> base_link transform

LiDAR
-> SLAM Toolbox during mapping OR AMCL/localization in a known map
    -> map -> odom transform

Nav2
-> costmaps + planner + controller + behavior tree
    -> /cmd_vel
    -> base controller / MCU

```

17.2 Map workflow

Stage	Action	Output
Mapping	Run SLAM Toolbox or mapping stack while teleoperating slowly.	Map + pose graph + bag.
Map cleanup	Remove temporary obstacles, validate scale, define keepout/speed zones if needed.	Clean map files and metadata.
Localization	Run AMCL or SLAM localization mode against map.	Stable map->odom transform.
Navigation tuning	Tune costmaps, inflation, controller, velocity limits.	Repeatable waypoint performance.
Regression	Replay bags and simulation scenarios after changes.	Confidence that updates did not break navigation.

17.3 Tuning priorities

Problem	Likely causes	What to inspect
Robot oscillates near obstacles	Inflation radius, controller gains, footprint wrong.	Costmap, footprint, controller logs.
Robot drifts in map	Wheel odometry scale, IMU orientation, LiDAR motion distortion.	tf tree, /odom, bag replay.
Planner refuses narrow spaces	Footprint too large, costmap resolution, inflation too high.	RViz costmap overlay.
Robot clips obstacles	Footprint too small, sensor blind spots, stale data, speed too high.	Footprint, scan data, costmap update rate.
Frequent recovery behaviors	Localization uncertainty, obstacle persistence, controller limits.	BT logs, costmaps, localization covariance.

17.4 Behavior trees

Nav2 uses behavior trees to orchestrate navigation actions, services, and recovery behavior. The behavior tree files are intentionally configurable; you can modify them for application-specific logic such as clearing costmaps, backing up, waiting for a door, docking, or retrying with alternate planners [S8].

18. Manipulation stack: MoveIt 2, grippers, and perception

MoveIt 2 should be introduced first in simulation or with the arm fixed to a bench. Once arm planning and collision geometry are stable, mount the arm on the mobile base and add the base geometry to the planning scene.

18.1 MoveIt integration checklist

Item	Why it matters
URDF links/joints	Kinematics, transforms, visualization.
Joint limits	Prevents commands outside arm capability.
Collision geometry	Fast, safe planning; avoid self and base collision.
SRDF	Planning groups, end effectors, disabled collision pairs.
Controllers	Maps planned trajectories to real/sim arm controller.
Planning scene	Objects, shelves, table, robot body, attached objects.
Gripper interface	Open/close/width/force actions or services.
Hand-eye calibration	Needed if camera guides grasping.

18.2 Gripper design

A gripper should match the task more than the arm. Parallel jaw is the best default. Suction is good for flat nonporous surfaces. Soft grippers help with uncertain objects but complicate modeling. Add force or current feedback if you need reliable grasp confirmation.

18.3 Mobile manipulator coordination

Task: pick object from shelf

1. Nav2 moves base to pre-grasp waypoint.
2. Base enters hold/brake state.
3. Perception updates object pose relative to camera/base.
4. Transform object pose into planning frame.
5. MoveIt plans arm trajectory with base collision geometry.
6. Gripper closes and verifies grasp.
7. Arm moves to safe carry pose.
8. Base navigation resumes with reduced speed if payload is carried.

18.4 Avoid early complexity

- Do not start with simultaneous base + arm trajectory optimization unless that is the research goal.
- Do not use a high-payload arm before verifying base stability.
- Do not trust visual meshes as collision meshes.
- Do not mount the camera on a flexible arm link unless you can calibrate and model it.
- Do not skip a stationary arm test rig. It is faster and safer than debugging on the mobile base.

19. Perception and AI pipeline

Perception is where compute gets consumed quickly. A single raw camera stream can exceed hundreds of MB/s before compression, neural networks, logging, or visualization. The design goal is to process data near the source, publish only useful products when possible, and record enough raw data for debugging.

19.1 Perception layers

Layer	Examples	Output
Sensor drivers	Camera, LiDAR, IMU, depth sensor.	Raw images, scans, point clouds, camera info.
Preprocessing	Rectification, depth alignment, voxel filtering, downsampling.	Cleaned and lower-rate data.
Detection/estimation	Objects, fiducials, people, obstacles, grasp poses.	Semantic messages, poses, masks, bounding boxes.
Fusion/tracking	Combine camera, LiDAR, odometry, map context.	Tracked objects and obstacle layers.
Behavior interface	Task planner consumes semantic state.	Actions, goals, constraints.

19.2 Bandwidth controls

- Downsample or crop early. Do not publish high-resolution raw images to every node by default.
- Use compressed image transport for remote visualization when latency allows.
- Avoid recording every raw stream all the time; define bag profiles.
- Use best-effort QoS for high-rate sensor streams where stale data is worse than missing data.
- Prefer C++ or accelerated pipelines for high-rate image and point-cloud processing.
- Separate debug visualization from deployed runtime. RViz and full-resolution Foxglove panels are not free.

19.3 AI model deployment

Need	Possible path	Caution
Object detection	YOLO/RT-DETR/DetectNet class model via PyTorch/ONNX/TensorRT depending compute.	Version models and labels. Measure latency on robot, not laptop.
Depth-based obstacle extraction	RGB-D point cloud filtering or depth image thresholding.	Depth cameras fail on reflective/transparent surfaces.
Fiducial localization	AprilTags/ArUco with calibrated camera.	Lighting, motion blur, and mounting rigidity matter.
Grasp pose estimation	Depth camera + segmentation + grasp planner.	Hard problem; first constrain objects and workspace.
People awareness	Camera detection plus conservative navigation constraints.	Do not treat non-safety-rated AI as a safety system.

20. Logging, observability, and debugging

A robot you cannot observe is a robot you cannot improve. ROS 2 gives you CLI tools, RViz, rosbag2, tracing, diagnostics, and third-party observability tools. rosbag2 records ROS 2 communication for replay and debugging [S13], while ros2_tracing provides lower-level tracing instrumentation for Linux-based performance analysis [S14].

20.1 Bag recording profiles

Profile	Topics	When to use
navigation_light	/tf, /odom, /scan, /cmd_vel, /diagnostics, Nav2 status	Most drive tests.
perception_debug	Selected images/depth/point clouds plus detections	Short perception sessions only.
full_failure	All core topics for 1-5 minutes around fault	Triggered by fault or manual command.
calibration	Camera info, images, tf, target detections	Sensor/hand-eye calibration.
power_test	Battery, rails, motor current, cmd_vel, odom	Runtime, acceleration, thermal testing.

20.2 Operator dashboards

Use RViz for deep ROS visualization and Foxglove for cross-platform live visualization, logs, plots, and browser/desktop workflows. Foxglove documents ROS 2 connections through Foxglove Bridge [S15].

20.3 Diagnostics contract

Every hardware-facing node should report:

- OK/WARN/ERROR/STALE level
- firmware/driver version
- communication status and latency
- voltage/current/temperature where available
- last command time and watchdog state
- fault code and recovery instructions

Every mission log should include:

- Git commit or release version
- robot serial/config profile
- map version
- parameter bundle version
- bag profile name

21. Development operations, deployment, and OTA

Robotics development gets slow when every change requires manual SSH, hand-edited configs, and mystery state on the robot. Treat the robot like a small deployable system: version the code, version the parameters, automate builds, automate launch, log releases, and keep a rollback path.

21.1 Release model

```
main branch:
- Builds in CI
- Passes unit tests and lint
- Can run simulation smoke test

develop branches:
- Feature-specific
- Short-lived
- Merged only after sim or bench test evidence

release tag:
robot-v0.3.0
includes:
  code commit
  parameter bundle
  firmware versions
  map version
  deployment notes
```

21.2 Systemd bringup pattern

```
# /etc/systemd/system/my-robot-bringup.service
[Unit]
Description=My Robot ROS 2 bringup
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=robot
Environment=ROS_DOMAIN_ID=42
Environment=RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
ExecStart=/home/robot/robot_ws/install/my_robot_bringup/bin/start_robot.sh
Restart=on-failure
RestartSec=3

[Install]
WantedBy=multi-user.target
```

21.3 Docker and dev containers

Docker can make development repeatable across Mac, Windows, and Linux. Official Docker guidance covers ROS 2 base images and dev-container workflows [S16]. Use containers carefully on the robot: GPU access, device permissions, host networking, DDS discovery, time sync, and real-time scheduling must be configured deliberately.

21.4 OTA options

Update scope	Simple prototype	More robust product-like path
ROS app	rsync or git pull from tagged release, then colcon build.	Debian packages or container images with versioned parameters.
Linux OS	Manual apt updates in lab.	Mender/RAUC A/B OTA with rollback [S25, S26].
MCU firmware	USB/SWD/DFU flash script.	Signed bootloader/OTA with physical recovery header [S27, S28].
Maps/configs	Copy files manually.	Versioned config bundle with rollback and metadata.

21.5 Remote access

- Use SSH keys, not passwords, and keep a local emergency account.
- Use WireGuard/Tailscale-like private networking only if you understand security implications.
- Never allow remote update to arm the robot automatically after reboot.
- After any OTA, robot should boot into safe idle and require explicit enable/arming.
- Keep a physical recovery path: keyboard/monitor, serial console, recovery image, MCU debug header.

22. Safety, reliability, and failure-mode design

Safety is not a package you add later. You need a hardware stop path, firmware safe states, conservative speed limits, current limits, physical guards, tests, and operational procedures. If the robot will operate near people or in commercial settings, research applicable standards early. For industrial robots and applications, ISO 10218-1:2025 and ISO 10218-2:2025 are current robot safety standards; ISO 3691-4:2023 addresses driverless industrial trucks including AMR-like systems; IEC 61508 is a broad functional safety reference [S33].

22.1 Minimum safety architecture

Physical E-stop button(s)

- > safety relay or motor-enable interruption
- > motor drivers disabled or commanded safe stop
- > MCU sees E-stop state
- > ROS diagnostics report E-stop state

Independent watchdogs:

- MCU command timeout stops motors
- Linux node heartbeat monitored by MCU/bridge
- Battery/BMS fault inhibits motion
- Arm fault prevents base-arm coordinated task

22.2 Failure modes and controls

Failure	Likely cause	Control
Robot keeps moving after command loss	No watchdog or stale command accepted.	MCU command timeout and default stop.
Linux brownout	Motor surge pulls compute rail down.	Separate DC/DC rails, power logging, wiring margin.
False localization	Bad map, reflective LiDAR, wheel slip.	Speed limits, localization confidence checks, obstacle sensors.
Arm collides with base/sensor	Missing collision geometry or frame error.	Movelt collision model, joint limits, slow commissioning.
Battery overcurrent/thermal	Undersized pack/BMS/wires.	Peak-current analysis, fuses, BMS telemetry, thermal tests.
OTA bricks robot	No rollback or interrupted update.	A/B update, physical recovery, update in safe idle only.
AI misses a person	Model limitations, lighting, occlusion.	Do not use non-safety-rated AI as safety stop; use conservative navigation and sensors.

22.3 Commissioning rules

- First motor tests with wheels off ground.
- First drive tests with speed/current limits and a person holding E-stop.
- First arm tests on a bench, then mounted but with base disabled.
- First navigation tests in a cleared area with soft obstacles.
- Raise speed limits only after logs show stable control and localization.
- Document every fault and fix root causes before adding more autonomy.

23. Build roadmap: from blank slate to autonomous mobile manipulator

This phased roadmap lets you start immediately without knowing every final hardware choice. Each phase produces an artifact, not just a learning experience.

23.1 Phase plan

Phase	Goal	Exit criteria
0 - Requirements	Write requirements, mission profile, budget envelope, safety assumptions.	requirements.yaml and initial risk list committed.
1 - Dev environment	Install ROS 2 baseline, create workspace, CI/lint, dev container or Linux setup.	Turtlesim/demo + empty robot workspace builds reproducibly.
2 - Robot model	Create URDF/Xacro, frame tree, placeholder sensors/arm/base.	RViz shows correct robot frames; no frame-name ambiguity.
3 - Simulation base	Differential drive in Gazebo/Webots/Isaac; simulated LiDAR and odom.	Teleop simulated robot; /tf, /scan, /odom valid.
4 - Navigation simulation	Nav2 + SLAM/localization in a simple world.	Robot completes waypoint route in simulation and records bag.
5 - Electronics bench	MCU, motor driver, encoder, power rail, E-stop on bench.	Wheel spins under closed-loop control; watchdog stops motor.
6 - Rolling base	Mount motors, battery, compute, MCU, E-stop, encoders.	Teleop base at low speed with odometry and diagnostics.
7 - Real sensors	Mount LiDAR/camera/IMU and calibrate frames.	Sensor bag shows correct frame IDs, timestamps, data rates.
8 - Real navigation	Map, localize, tune Nav2 on real base.	Repeatable autonomous route at conservative speed.
9 - Arm bench	Bring up arm, gripper, Movelt on bench or fixed mount.	Plan and execute safe trajectories; E-stop behavior tested.
10 - Arm on base	Mount arm and update URDF/collision/power/stability.	Stationary mobile base can manipulate safely.
11 - Perception task	Detect object/marker and transform pose to robot frame.	Bag replay and live test produce stable object poses.
12 - Integrated behavior	Behavior tree/task executive coordinates navigation, perception, arm.	End-to-end demo with logs and documented safety envelope.

23.2 First 30 days

Week	Work
Week 1	Pick ROS baseline, set up Ubuntu/WSL/remote dev, create repository, write requirements and interface contract draft.
Week 2	Build URDF/Xacro skeleton: base, wheels, LiDAR, camera, IMU, arm placeholder. View in RViz.
Week 3	Add Gazebo simulation with differential drive and LiDAR. Start Nav2 tutorials and record simulated bags.
Week 4	Create hardware decision matrix. Shortlist compute, drive base, motor driver, LiDAR, camera, battery. Do not buy until power/torque/sensor mounting checks are complete.

24. A concrete starter configuration

The following is a practical starting architecture that maximizes learning and minimizes early risk. It is not the only correct design, but it is a strong default for a first serious autonomous robot with future arms.

Subsystem	Starter choice	Why
Drive base	Differential drive, 24 V class, wheel encoders, conservative speed.	Fastest path to Nav2 autonomy and easy simulation.
Main compute	x86 mini PC with NVMe or Jetson Orin Nano Super-class device.	Enough headroom for Nav2, perception, logs; Jetson if AI is central.

Subsystem	Starter choice	Why
MCU	STM32/Teensy-class drive controller, CAN or USB serial initially.	Deterministic encoder/motor/watchdog control.
LiDAR	2D LiDAR with ROS 2 driver support.	Best first mapping/navigation sensor.
Camera	Front RGB-D/stereo camera with ROS 2 driver support.	Useful for object detection, depth, teleop, and later grasping.
IMU	ROS-compatible 6/9-axis IMU mounted near base center.	Improves odometry and diagnostic insight.
Arm	Off-the-shelf ROS 2-supported 6-DoF arm if budget allows; otherwise leave arm placeholder in model.	Avoids custom arm complexity until base autonomy works.
Battery	LiFePO4 or Li-ion pack with BMS, fused branches, DC/DC rails.	Robust power and safer integration.
Software	ROS 2 Jazzy or Lyrical, Nav2, MoveIt 2, ros2_control, SLAM Toolbox, robot_localization, Gazebo.	Mainstream ROS 2 architecture with simulation path.

24.1 Why not start with everything at once?

Because every subsystem can hide faults in the others. If navigation fails while the arm is installed, you will not know whether the cause is power sag, LiDAR placement, bad odometry, frame errors, CPU load, or unstable center of gravity. Build in layers, keep logs, and maintain a working baseline.

A. Appendix: hardware scoring matrix

Score every candidate 1-5 for each criterion. Reject parts with a score of 1 in safety, power, mounting, or ROS support unless you intentionally want that part to be a research problem.

Criterion	Weight	What 5 means
ROS 2 driver/support	High	Maintained driver for your target distro or clean documented API.
Electrical fit	High	Voltage/current/connector/EMI requirements fit your power tree.
Mechanical fit	High	Mounting, mass, center of gravity, cable routing are practical.
Simulation model	Medium	URDF/SDF/CAD model or enough dimensions to create one.
Safety controls	High	Enable line, fault reporting, limits, watchdog, predictable stop behavior.
Documentation	Medium	Good datasheets, protocol docs, examples, active support.
Availability	Medium	Can buy replacements; not a one-off surplus part.
Cost	Medium	Fits budget with spares and required accessories.
Learning value	Low/medium	Teaches skills you actually need for the project.

B. Appendix: software templates

B.1 Top-level launch shape

```
# robot.launch.py shape, not complete code
Launch arguments:
  use_sim_time
  robot_model
  start_nav
  start_arm
  start_perception
  namespace

Launch includes:
  robot_state_publisher
  ros2_control controller manager or hardware bridge
  sensors.launch.py
  localization.launch.py
  navigation.launch.py
  manipulation.launch.py
  diagnostics.launch.py

Rules:
  - Same topic names in sim and hardware.
  - Hardware drivers can be swapped by launch arguments.
  - Parameters live in YAML, not hard-coded Python launch files.
```

B.2 Udev naming rule example

```
# /etc/udev/rules.d/99-my-robot.rules
# Replace vendor/product/serial with real values from udevadm info.
SUBSYSTEM=="tty", ATTRS{idVendor}=="1234", ATTRS{idProduct}=="5678",
SYMLINK+="my_robot_drive_mcu", MODE="0660", GROUP="dialout"
```

B.3 Bag recording command examples

```
# Navigation light profile
ros2 bag record -o bags/nav_$(date +%Y%m%d_%H%M%S) /tf /tf_static /odom /scan /cmd_vel
/diagnostics

# Perception short profile
ros2 bag record -o bags/perception_$(date +%Y%m%d_%H%M%S) /tf /tf_static
/camera/color/image_raw /camera/depth/image_rect_raw /detections /diagnostics
```

C. Appendix: calculations worksheet

Worksheet item	Formula / data needed	Design output
Total mass	Chassis + battery + compute + motors + sensors + arm + payload + 20 percent growth	Mass target for motors and frame.
Wheel torque	Use formula in Section 6.3	Motor and gearbox shortlist.
Wheel RPM	$v / \text{circumference} \times 60$	Gear ratio and motor speed.
Battery Wh	Average W x hours / usable fraction / efficiency	Battery pack size.
Peak current	Peak W / pack voltage	BMS, fuse, wire, connector size.
Compute power	CPU/GPU worst case + sensors + storage + fans	DC/DC rail size.
Arm tipping	Moment from arm + payload versus wheelbase/support polygon	Base width, ballast, speed limits, outriggers.
Sensor occlusion	Sensor FOV versus robot body geometry	Sensor height and mounting location.

D. Appendix: commissioning checklists

D.1 Before applying battery power

- Continuity check power rails and polarity.
- Verify fuses and wire gauges.
- Confirm E-stop interrupts motor enable/power.
- Confirm no exposed conductors or loose hardware.
- Power logic rails from bench supply first if possible.
- Current-limit first power-up.

D.2 Before first autonomous drive

- Teleop works at low speed.
- /tf tree has no missing frames or wrong parent-child relationships.
- /odom direction matches real motion.
- LiDAR scan aligns with robot body in RViz.
- E-stop tested during motion.
- Bag recording profile ready.
- Nav2 speed limits conservative.

D.3 Before arm motion on base

- Arm works on bench or with base disabled.
- MoveIt collision model includes base, sensor mast, and arm mount.
- Base is physically stable at worst-case arm reach.
- Cable routing checked through full joint range.
- Gripper force/current limits set.
- E-stop behavior tested with arm and base controllers.

E. Appendix: research trail and source register

Use this section as the first set of places to research further. The source register emphasizes official documentation, maintained project documentation, and academic papers. All source choices should be rechecked when you select final hardware because driver support can change quickly.

E.1 Search terms to continue research

Topic	Useful search terms
ROS distro support	ROS 2 Lyrical supported platforms, REP 2000, ROS 2 Jazzy release notes
Nav2	Nav2 behavior trees, Nav2 costmap tuning, Nav2 robot_localization odometry
Simulation	ROS 2 Gazebo ros_gz, gz_ros2_control, URDF to SDF, Gazebo Harmonic Jetty ROS pairing
Arms	Movelt 2 setup assistant, FollowJointTrajectory ROS 2, xarm_ros2 Jazzy, ros2_kortex
Embedded	micro-ROS FreeRTOS Zephyr NuttX, ROS 2 CANopen, ros2_control hardware interface
Perception	ROS 2 image_transport, PointCloud2 voxel filter, Isaac ROS, DepthAI ROS driver
OTA	Mender A/B update, RAUC bundle, ESP-IDF OTA partition, PlatformIO remote upload
Safety	ISO 10218-1 2025, ISO 3691-4 2023, IEC 61508, AMR safety risk assessment

- S1.** ROS 2 Lyrical Luth release and installation docs. <https://docs.ros.org/en/rolling/Releases/Release-Lyrical-Luth.html> ; <https://docs.ros.org/en/lyrical/Installation.html>
- S2.** ROS 2 Jazzy Jalisco release and installation docs. <https://docs.ros.org/en/rolling/Releases/Release-Jazzy-Jalisco.html> ; <https://docs.ros.org/en/jazzy/Installation.html>
- S3.** REP 2000 - ROS 2 Releases and Target Platforms. <https://www.ros.org/reps/rep-2000.html>
- S4.** ROS 2 interfaces: topics, services, and actions. <https://docs.ros.org/en/rolling/Concepts/Basic/Interfaces-Topics-Services-Actions.html>
- S5.** ROS 2 Quality of Service settings. <https://docs.ros.org/en/rolling/Concepts/Intermediate/About-Quality-of-Service-Settings.html>
- S6.** ROS 2 real-time programming tutorial. <https://docs.ros.org/en/rolling/Tutorials/Demos/Real-Time-Programming.html>
- S7.** Macenski et al., Robot Operating System 2: Design, architecture, and uses in the wild. <https://arxiv.org/abs/2211.07752> ; DOI: 10.1126/scirobotics.abm6074
- S8.** Nav2 documentation. <https://docs.nav2.org/>
- S9.** MoveIt 2 documentation. <https://moveit.picknik.ai/main/index.html>
- S10.** ros2_control documentation. <https://control.ros.org/rolling/index.html>
- S11.** micro-ROS features and architecture. <https://micro.vulcanexus.org/docs/overview/features/>
- S12.** Gazebo ROS 2 installation and integration. https://gazebo-sim.org/docs/latest/ros_installation/ ; https://gazebo-sim.org/docs/latest/ros2_integration/
- S13.** ROS 2 bag recording and playback. <https://docs.ros.org/en/rolling/Tutorials/Beginner-CLI-Tools/Recording-And-Playing-Back-Data/Recording-And-Playing-Back-Data.html>
- S14.** ros2_tracing. https://github.com/ros2/ros2_tracing ; <https://docs.ros.org/en/rolling/Tutorials/Advanced/ROS2-Tracing-Trace-and-Analyze.html>
- S15.** Foxglove ROS 2 documentation. <https://docs.foxglove.dev/docs/getting-started/frameworks/ros2>
- S16.** Docker guide for ROS 2 development. <https://docs.docker.com/guides/ros2/>
- S17.** ROS 2 Windows installation docs. <https://docs.ros.org/en/jazzy/Installation/Windows-Install-Binary.html>
- S18.** ROS 2 macOS source installation docs. <https://docs.ros.org/en/jazzy/Installation/Alternatives/macOS-Development-Setup.html>
- S19.** NVIDIA Jetson Orin Nano Super Developer Kit. <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/nano-super-developer-kit/>
- S20.** Raspberry Pi 5 product specifications. <https://www.raspberrypi.com/products/raspberry-pi-5/>
- S21.** Luxonis DepthAI ROS Driver. <https://docs.luxonis.com/software-v3/depthai/ros/driver/>
- S22.** OrbbecSDK ROS2 documentation. https://orbbec.github.io/OrbbecSDK_ROS2_Docs/
- S23.** Example LiDAR ROS 2 drivers. https://github.com/Slamtec/sllidar_ros2 ; https://github.com/Hokuyo-aut/urg_node2 ; https://github.com/SICKAG/sick_scan_xd ; <https://github.com/ouster-lidar/ouster-ros>
- S24.** Example robot arm and actuator ROS 2 support. https://github.com/xArm-Developer/xarm_ros2 ; https://github.com/elephantrobotics/mycobot_ros2 ; https://github.com/Kinovarobotics/ros2_kortex ; https://manual.robotis.com/docs/en/software/dynamixel/dynamixel_sdk/overview/
- S25.** Mender embedded Linux OTA. <https://mender.io/platforms/ota-updates-for-embedded-linux-devices>
- S26.** RAUC embedded Linux update framework. <https://rauc.readthedocs.io/>
- S27.** PlatformIO Remote Development. <https://docs.platformio.org/en/latest/plus/pio-remote.html>
- S28.** ESP-IDF OTA updates. <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/ota.html>
- S29.** Macenski and Jambrecic, SLAM Toolbox: SLAM for the dynamic world. <https://joss.theoj.org/papers/10.21105/joss.02783>

- S30.** Macenski et al., The Marathon 2: A Navigation System. <https://arxiv.org/abs/2003.00368>
- S31.** Coleman et al., Reducing the Barrier to Entry of Complex Robotic Software: a MoveIt! Case Study. <https://arxiv.org/abs/1404.3785>
- S32.** Staschulat et al., Budget-based real-time Executor for Micro-ROS. <https://arxiv.org/abs/2105.05590>
- S33.** Safety standards starting points. <https://www.iso.org/standard/73933.html> ; <https://www.iso.org/standard/73934.html> ; <https://www.iso.org/standard/83545.html> ; <https://www.iec.ch/functional-safety>
- S34.** NVIDIA Isaac Sim ROS 2 bridge documentation. https://docs.isaacsim.omniverse.nvidia.com/latest/ros2_tutorials/ros2_landing_page.html
- S35.** Webots ROS 2 documentation. <https://docs.ros.org/en/lyrical/Tutorials/Advanced/Simulators/Webots/Simulation-Webots.html>
- S36.** NVIDIA Isaac ROS documentation. <https://nvidia-isaac-ros.github.io/>
- S37.** mujoco_ros2_control. https://github.com/ros-controls/mujoco_ros2_control

End of roadmap. Your next action is to create the repository skeleton, fill the requirements matrix, and build the first URDF/Xacro model before buying major hardware.